

Array

****You do not need to write the Array class, 2D Array class, printArray() etc. functions unless specified in the problem.****

****Just design the function specified in the problem.****

****No need to write driver code.****

****This document is a compilation of previous semester's exam (midterm/final) problems and lab quiz problems. Try to practice relevant problems from online judges as well.****

1.

- I. Suppose you are given a multi-dimensional array with dimension 5x5x3. What is the multidimensional index for the linear index 65?

- II. **Complete** the function **compress_matrix** that takes a 2D array as a parameter and return a new compressed 2D array. In the given array the number of rows and columns will always be even. **Compressing a matrix means grouping elements in 2x2 blocks and sums the elements within each block. Check the sample input output for further clarification.**

Hint: Generally the block consists of the (i,j), (i+1,j), (i,j+1) and (i+1, j+1) elements for 2x2 blocks.

You cannot use any built-in function except len() and range(). You can use the np variable to create an array.

Python Notation	Java Notation
import numpy as np def compress_matrix (mat): # To Do	public int[][] compress_matrix (int[][] mat) { // To Do }

Sample Input array	All Box (No need to create these arrays)	Returned Array	Explanation
[[1, 2, 3, 4], [5, 6, 7, 8], [1, 3, 5, 2], [-2, 0, 6, -3]]	[[1, 2], [[3, 4], [5, 6]] [[7, 8]] [[1, 3], [[5, 2], [-2, 0]] [[6, -3]]	[[14, 22], [2, 10]]	[[1+2+5+6, 3+4+7+8], [1+3+-2+0, 5+2+6+-3]]

2.

- I. Suppose you are given a multi-dimensional array with dimensions 4x5x4. What is the linear index of the multidimensional index [2][1][0]?
- II. Suppose you're working as a cryptographer for a secret intelligence agency. You are given an encrypted matrix as an input. You have to decrypt it after some processing of this matrix.

To decrypt this message efficiently, you have a task to construct a method called **sum_diff(matrix)** which takes an encrypted matrix as input and returns a decrypted linear array. The process of finding the decrypted linear array is given below:

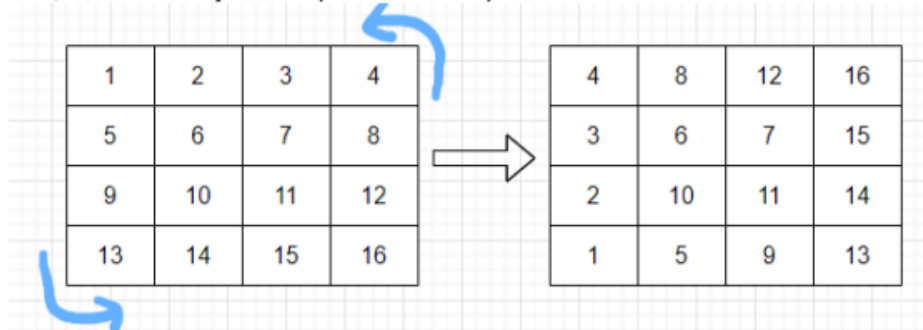
You have to find out the column-wise summations for each column and store the difference of subsequent column-wise summations in a new linear array.

Sample Input	Sample output	Explanation																			
<table><tr><td>1</td><td>3</td><td>1</td></tr><tr><td>6</td><td>4</td><td>2</td></tr><tr><td>5</td><td>1</td><td>7</td></tr><tr><td>9</td><td>3</td><td>3</td></tr><tr><td>8</td><td>5</td><td>4</td></tr></table>	1	3	1	6	4	2	5	1	7	9	3	3	8	5	4	<table><tr><td>-13</td><td>1</td></tr></table>	-13	1	Sum of 0th column = 29 Sum of 1st column = 16 Sum of 2nd column = 17 Therefore, the size of the resulting array is 2 and the array is: <table><tr><td>16-29 = -13</td><td>17-16 = 1</td></tr></table>	16-29 = -13	17-16 = 1
1	3	1																			
6	4	2																			
5	1	7																			
9	3	3																			
8	5	4																			
-13	1																				
16-29 = -13	17-16 = 1																				

Python Notation	Java Notation
<pre>def sum_diff(matrix): # To Do</pre>	<pre>public int[] sum_diff(matrix) { // To Do }</pre>

3.

- I. Suppose you are given a multi-dimensional array with dimensions 4x3x2x6. What is the corresponding linear index of the multidimensional index [2][1][0][4]?
- II. Bob has already learnt about 2D matrices and its operations in the data structures course. However, he is currently facing problems in rotating a 2D square matrix. As such, he has asked you to help him solve the problem.



You should rotate the original input matrix's outer rows and columns in left, down, right, top manner without affecting the internal cells; **instead of creating a new matrix**. Here, as you can see, if you rotate the given square matrix **anticlockwise**, 1st row will be 1st column, 1st column will be last row, last row will be last column and last column will be 1st row. The elements of the internal cells will be unchanged.

So given a square matrix of size $n \times n$, your task is to complete the given method **rotate(matrix)** that takes a 2D square matrix as an input and returns the rotated matrix using **in-place** algorithm.

Hints of in-place: Rotating by one element at a time instead of an entire row/column should eliminate the need for any supporting data structures.

Sample Input	Sample Output
<pre> 1 2 3 ----- 5 6 7 ----- 9 10 11 ----- </pre>	<pre> 3 7 11 ----- 2 6 10 ----- 1 5 9 ----- </pre>

Sample Input	Sample Output
<pre> 1 2 3 4 ----- 5 6 7 8 ----- 9 10 11 12 ----- 13 14 15 16 ----- </pre>	<pre> 4 8 12 16 ----- 3 6 7 15 ----- 2 10 11 14 ----- 1 5 9 13 ----- </pre>

Python Notation	Java Notation
<pre> def rotate(matrix): # To Do </pre>	<pre> public int[][] rotate(int[][] matrix) { // To Do } </pre>

4.

- I. Suppose you are given a multi-dimensional array with dimensions 3x4x2. What is the linear index of the multidimensional index [2][1][0]?
- II. You have hidden a password into a square matrix for your friend. Only you know that the password can be achieved through traversing the array in a zigzag fashion. Now help your friend develop a function/method that will take the 2D array as input and print the password in the output console.
Complete the function `zig_zag` which will take a NxN 2D array and print all the values in a zigzag pattern. Each value will be separated by a space. Assume the function will always receive a NxN 2D array. Your code should work for any NxN array.

Input Array	Output	Zigzag traversal explanation
<pre> D B G S A G T S W U R N O H R O </pre>	<pre> D A B W G G O U T S H R S R N O </pre>	

Hint: You need to follow the arrowed line 1 to line 7 to find out the zigzag pattern. Use one loop to print the upper triangle (line 1 to 4) and another loop to print the lower triangle (line 5 to 7).

Python Notation	Java Notation
<pre> def zig_zag(arr): # To Do </pre>	<pre> public void zig_zag(int[][] arr) { // To Do } </pre>

5.

You are given an $n \times n$ 2D matrix representing an image. Rotate the image by **90 degrees (clockwise)**. You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. You can assume that the input matrix will always be a square matrix.

Sample Input	Sample Output
<pre>[[1, 2, 3], [4, 5, 6], [7, 8, 9]]</pre>	<pre>[[7, 4, 1], [8, 5, 2], [9, 6, 3]]</pre>
<pre>[[5, 1, 9,11], [2, 4, 8,10], [13, 3, 6, 7], [15,14,12,16]]</pre>	<pre>[[15,13, 2, 5], [14, 3, 4, 1], [12, 6, 8, 9], [16, 7,10,11]]</pre>

6.

Question: You are given an $n \times n$ 2D matrix representing an image. Rotate the image by **90 degrees (counter-clockwise)**. You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. You can assume that the input matrix will always be a square matrix.

Sample Input	Sample Output
<pre>[[1, 2, 3], [4, 5, 6], [7, 8, 9]]</pre>	<pre>[[3, 6, 9], [2, 5, 8], [1, 4, 7]]</pre>
<pre>[[5, 1, 9, 11], [2, 4, 8, 10], [13, 3, 6, 7], [15, 14, 12, 16]]</pre>	<pre>[[11, 10, 7, 16], [9, 8, 6, 12], [1, 4, 3, 14], [5, 2, 13, 15]]</pre>

7.

Question: You have joined as an intern under a renowned cyber specialist and have to assist her in locating a murderer. After staring at the cryptic messages sent by the murderer for hours, you have finally figured out the pattern: your main clue is on the primary diagonal, that is, you will start comparing the alphabets from the top left and bottom right till you reach the end point of the diagonal and the mismatched alphabets are your potential apartment names.

The murderer thinks that the law enforcement is incapable which is why this person sent a hint with the latest message. The hint states that upon decoding this message, you will be able to pinpoint the possible apartment numbers the murderer may be hiding in. The clock is ticking and you have 20 minutes to bring justice to the victims.

Exam Instructions: The 2D array will be a squared one.

The order of the apartment names do not matter since you have to search each one anyway.

Sample Input	Sample Output																									
<table><tr><td>A</td><td>D</td><td>M</td><td>Q</td></tr><tr><td>E</td><td>S</td><td>Y</td><td>K</td></tr><tr><td>J</td><td>F</td><td>O</td><td>L</td></tr><tr><td>P</td><td>X</td><td>J</td><td>A</td></tr></table>	A	D	M	Q	E	S	Y	K	J	F	O	L	P	X	J	A	Possible Apartment Names: S O									
A	D	M	Q																							
E	S	Y	K																							
J	F	O	L																							
P	X	J	A																							
<table><tr><td>A</td><td>D</td><td>M</td><td>Q</td><td>F</td></tr><tr><td>E</td><td>S</td><td>Y</td><td>K</td><td>W</td></tr><tr><td>J</td><td>F</td><td>O</td><td>L</td><td>T</td></tr><tr><td>P</td><td>X</td><td>J</td><td>S</td><td>Y</td></tr><tr><td>V</td><td>R</td><td>K</td><td>G</td><td>P</td></tr></table>	A	D	M	Q	F	E	S	Y	K	W	J	F	O	L	T	P	X	J	S	Y	V	R	K	G	P	Possible Apartment Names: A P O
A	D	M	Q	F																						
E	S	Y	K	W																						
J	F	O	L	T																						
P	X	J	S	Y																						
V	R	K	G	P																						

```
//python
def decodeMessage(codeword):
    #To Do
    pass
```

```
//java
public static void decodeMessage(String[][] codeword){
    //To Do
}
```

8.

Question: You have joined as an intern under a renowned cyber specialist and have to assist her in locating a murderer. After staring at the cryptic messages sent by the murderer for hours, you have finally figured out the pattern: your main clue is on the secondary diagonal, that is, you will start comparing the alphabets from the top right and bottom left till you reach the end point of the diagonal and the mismatched alphabets are your potential apartment names.

The murderer thinks that the law enforcement is incapable which is why this person sent a hint with the latest message. The hint states that upon decoding this message, you will be able to pinpoint the possible apartment numbers the murderer may be hiding in. The clock is ticking and you have 20 minutes to bring justice to the victims.

Exam Instructions: The 2D array will be a squared one.

The order of the apartment names do not matter since you have to search each one anyway.

Sample Input				Sample Output																									
<table><tr><td>A</td><td>D</td><td>M</td><td>Q</td></tr><tr><td>E</td><td>S</td><td>Y</td><td>K</td></tr><tr><td>J</td><td>Y</td><td>O</td><td>L</td></tr><tr><td>P</td><td>X</td><td>J</td><td>A</td></tr></table>				A	D	M	Q	E	S	Y	K	J	Y	O	L	P	X	J	A	Possible Apartment Names: Q P									
A	D	M	Q																										
E	S	Y	K																										
J	Y	O	L																										
P	X	J	A																										
<table><tr><td>A</td><td>D</td><td>M</td><td>Q</td><td>F</td></tr><tr><td>E</td><td>S</td><td>Y</td><td>K</td><td>W</td></tr><tr><td>J</td><td>F</td><td>O</td><td>L</td><td>T</td></tr><tr><td>P</td><td>X</td><td>J</td><td>S</td><td>Y</td></tr><tr><td>F</td><td>R</td><td>K</td><td>G</td><td>P</td></tr></table>				A	D	M	Q	F	E	S	Y	K	W	J	F	O	L	T	P	X	J	S	Y	F	R	K	G	P	Possible Apartment Names: K O X
A	D	M	Q	F																									
E	S	Y	K	W																									
J	F	O	L	T																									
P	X	J	S	Y																									
F	R	K	G	P																									

```
//python
def decodeMessage(codeword):
    #To Do
    pass
}
```

```
//java
public static void decodeMessage(String[][] codeword){
    //To Do
}
```


9.

Find The Queen's Age 👑

In a kingdom, the ages of noble members are recorded in an $N \times M$ matrix, where each cell represents a noble's age. The Queen's age is determined by identifying *the row with the smallest sum of ages* and *the column with the smallest sum of ages*, then *selecting the value at their intersection*. It is guaranteed that there is only one such minimum row and one such minimum column. Now, write a function, **findQueenAge(matrix)**, that takes the matrix as a parameter and returns the Queen's age.

Sample Given Matrix	Sample Output	Explanation												
<table><tr><td>50</td><td>45</td><td>90</td><td>35</td></tr><tr><td>70</td><td>80</td><td>60</td><td>95</td></tr><tr><td>40</td><td>30</td><td>25</td><td>75</td></tr></table>	50	45	90	35	70	80	60	95	40	30	25	75	30	• Smallest row sum is found in the 3rd row. • Smallest column sum is found in the 2nd column. • Finally, Queen's Age is found at the intersection of the 3rd row and 2nd column.
50	45	90	35											
70	80	60	95											
40	30	25	75											

[You are not allowed to use any built-in functions]

10.

Find The Strongest Energy Source ⚡

In a futuristic city, an $N \times N$ energy matrix records the power levels of energy sources. The strongest energy source is determined by finding *the column with the largest sum of power levels* and *the row with the largest sum of power levels*, then *selecting the value at their intersection*. There is guaranteed to be only one such maximum column and one such maximum row. Now, write a function, **findStrongSource(matrix)**, that takes a matrix as a parameter and returns the strongest energy source.

Sample Given Matrix	Sample Output	Explanation									
<table><tr><td>5</td><td>3</td><td>6</td></tr><tr><td>7</td><td>4</td><td>9</td></tr><tr><td>1</td><td>2</td><td>8</td></tr></table>	5	3	6	7	4	9	1	2	8	9	• Largest column sum is found in the 3rd column. • Largest row sum is found in the 2nd row. • Finally, Strong Source is found at the intersection of the 3rd column and 2nd row.
5	3	6									
7	4	9									
1	2	8									

[You are not allowed to use any built-in functions]

11.

Tasks from Lab 1 - [W Lab 3 - 2D Matrix.docx](#)

Tasks from Book - [PDF Practice Sheet 1 - Array.pdf](#)

12.

Suppose you are a matrix transformation expert working on a mission for a tech company. You are given a 2D array (matrix) that contains important data, and your task is to transform the given matrix into a **symmetric lower triangular matrix** by **adding all elements above the main diagonal to their symmetric counterparts** below the diagonal, then setting the elements above the diagonal to zero.

❑ You need to implement the function `toSymLowerTri()`, which takes the matrix as a parameter and returns the transformed matrix.

❑ *Note: Your code should handle square matrices of any size. For non-square matrices, the function should return the matrix unchanged. You are not allowed to create any new matrices.*

Sample Input	Sample Output	Explanation																		
matrix= <table><tr><td>8</td><td>2</td><td>1</td></tr><tr><td>3</td><td>5</td><td>4</td></tr><tr><td>6</td><td>9</td><td>7</td></tr></table>	8	2	1	3	5	4	6	9	7	<table><tr><td>8</td><td>0</td><td>0</td></tr><tr><td>5</td><td>5</td><td>0</td></tr><tr><td>7</td><td>13</td><td>7</td></tr></table>	8	0	0	5	5	0	7	13	7	The values above the diagonal are added to their symmetric counterparts: • 2 is added to its counterpart 3, resulting in 5. • 1 is added to its counterpart 6, resulting in 7. • 4 is added to its counterpart 9, resulting in 13. After the addition, the elements above the diagonal are set to zero.
8	2	1																		
3	5	4																		
6	9	7																		
8	0	0																		
5	5	0																		
7	13	7																		

13.

Suppose you are a matrix transformation expert working on a mission for a tech company. You are given a 2D array (matrix) that contains important data, and your task is to transform the given matrix into a **symmetric upper triangular matrix** by **adding all elements below the main diagonal to their symmetric counterparts** above the diagonal, then setting the elements below the diagonal to zero.

❑ You need to implement the function `toSymUpperTri()`, which takes the matrix as a parameter and returns the transformed matrix.

❑ *Note: Your code should handle square matrices of any size. For non-square matrices, the function should return the matrix unchanged. You are not allowed to create any new matrices.*

Sample Input	Sample Output	Explanation																		
matrix= <table><tr><td>8</td><td>2</td><td>1</td></tr><tr><td>3</td><td>5</td><td>4</td></tr><tr><td>6</td><td>9</td><td>7</td></tr></table>	8	2	1	3	5	4	6	9	7	<table><tr><td>8</td><td>5</td><td>7</td></tr><tr><td>0</td><td>5</td><td>13</td></tr><tr><td>0</td><td>0</td><td>7</td></tr></table>	8	5	7	0	5	13	0	0	7	The values below the diagonal are added to their symmetric counterparts: • 3 is added to its counterpart 2, resulting in 5. • 6 is added to its counterpart 1, resulting in 7. • 9 is added to its counterpart 4, resulting in 13. After the addition, the elements below the diagonal are set to zero.
8	2	1																		
3	5	4																		
6	9	7																		
8	5	7																		
0	5	13																		
0	0	7																		

14.

A university records student **quiz scores** in a **2D matrix**, where each **row** represents a different **student** and each **column** represents a different **quiz**. The university wants to analyze the **difficulty trend** of the quizzes. To do this, they need you to **decrypt the performance data** using the following steps:

1. **Compute the average score for each quiz .**
2. **Find the difference between consecutive quiz averages** to determine if quizzes are getting harder or easier.
3. **Return the list of differences**, which will be used to adjust future quiz difficulties.

Sample Input	Sample Output
85 90 78 88 85 80 75 95 85 92 88 82	[4.5 -8.25] Explanation: Quiz 0 avg = $(85 + 88 + 75 + 92) / 4 = 85.0$ Quiz 1 avg = $(90 + 85 + 95 + 88) / 4 = 89.5$ Quiz 2 avg = $(78 + 80 + 85 + 82) / 4 = 81.25$ Quiz 1 - Quiz 0 = $89.5 - 85.0 = 4.5$ Quiz 2 - Quiz 1 = $81.25 - 89.5 = -8.25$

15.

A company records **monthly sales** of multiple products in a **2D matrix**, where each **row** represents a different **month** and each **column** represents a different **product**. Each value represents the **sales revenue** for that product in that month.

To analyze **sales trends**, decrypt the data as follows:

1. **Compute the total sales for each product .**
2. **Find the percentage contribution of each product to total sales.**
3. **Compute the difference between consecutive product percentages.**

Sample Input	Sample Output
100 200 300 150 250 350 120 180 290	[13.40 15.98] Explanation Product 0: $100 + 150 + 120 = 370$ Product 1: $200 + 250 + 180 = 630$ Product 2: $300 + 350 + 290 = 940$ Total Sales = $370 + 630 + 940 = 1940$ Product 0: $(370 / 1940) * 100 = 19.07\%$ Product 1: $(630 / 1940) * 100 = 32.47\%$ Product 2: $(940 / 1940) * 100 = 48.45\%$ Product 1 - Product 0 = $32.47 - 19.07 = 13.40$ Product 2 - Product 1 = $48.45 - 32.47 = 15.98$

16.

Suppose you are an FBI field agent working on a mission. The HQ has sent you a secret **message** in 2D array format containing information about the **pin** code of a highly protected facility. Now, you need to decode the **pin** from the **message** using the method “**decodeMessage()**,” which takes the **message** as a **parameter** and returns a decoded **pin**.

Decoding procedure: You need to find out the **absolute difference** between the diagonal and counter-diagonal elements and create a new array containing the resultant values.

Notice: Consider the message being (MxM) 2D array, where “M” is **even**.

Sample Input	Sample Output
message= [[1, 2, 3, -4], [5, -6, 7, 8], [9, -10, 11, 12], [13, 14, 15, -16]]	[5, 13, 21, 29]
Explanation	
1st Diagonal element = 1 1st Counter Diagonal element = - 4 Absolute difference= $ 1 - (-4) = 5$	2nd Diagonal element = - 6 2nd Counter Diagonal element = 7 Absolute difference= $ -6 - 7 = 13$
3rd Diagonal element = 11 3rd Counter Diagonal element = - 10 Absolute difference= $ 11 - (-10) = 21$	4th Diagonal element = -16 4th Counter Diagonal element = 13 Absolute difference= $ -16 - 13 = 29$

17.

Suppose you are an FBI field agent working on a mission. The HQ has sent you a secret **message** in 2D array format containing information about the **pin** code of a highly protected facility. Now, you need to decode the **pin** from the **message** using the method “**decodeMessage()**,” which takes the **message** as a **parameter** and returns a decoded **pin**.

Decoding procedure: You need to find out the **absolute average** between the diagonal and counter-diagonal elements and create a new array containing the resultant values.

Notice: Consider the message being (MxM) 2D array, where “M” is **even**.

Sample Input	Sample Output
message= [[1, 2, 3, -4], [5, -6, 7, 8], [9, -10, 11, 12], [13, 14, 15, -16]]	[1.5, 0.5, 0.5, 1.5]
Explanation:	
1st Diagonal element = 1 1st Counter Diagonal element = - 4 Absolute average= $ 1+(-4) /2 = 1.5$	2nd Diagonal element = - 6 2nd Counter Diagonal element = 7 Absolute average= $ -6+7 /2 = 0.5$
3rd Diagonal element = 11 3rd Counter Diagonal element = - 10 Absolute average= $ 11+(-10) /2 = 0.5$	4th Diagonal element = -16 4th Counter Diagonal element = 13 Absolute average= $ -16+13 /2 = 1.5$

18.

Alex finds himself trapped inside a **magical dungeon** represented by an $m \times n$ grid of tiles. Each tile is either **black** (deadly) or **white** (safe). Each white tile contains a number representing the **power** you gain when stepping on it.

- You can only move through **white tiles** to stay alive.
- Your movement is **fixed**: You start at the **top-left corner (0,0)**, always a white tile.
- From any current tile, you follow this movement:
 - **Down** until you hit a black tile or the bottom edge.
 - **Right** one step.
 - **Up** until you hit a black tile or the top edge.
 - **Right** one step.
 - Repeat this **zigzag snake-like movement** until you cannot move right anymore.

Alex will collect the numbers from white tiles in this order and make a **singly linked list**. To solve this, Alex will use a function/method:

Zigzag_List(matrix) and it will return the **head** of the linked list. Now, help Alex to solve this.

Consider that the Node class is already provided with the **elem** being the node's value and **next** being the location of the next node.

Matrix 1:

1	2	5	8
4	9	1	0
7	3	4	9
6	8	3	2

Linked list:

0 → 2 → 4 → 5 → 9 → 8 → 7 → 1

Matrix 2:

1	2	5	8
4	9	1	0
7	3	4	9
6	8	3	2
2	5	3	7

Linked list:

0 → 2 → 3 → 4 → 5 → 9 → 8 → 2 → 7 → 1

19.

Question 1: CO1 [10 + 1 Points]

You are given a **9x5** dimensional 2D array which is used as a combination lock of a safe. You can independently rotate each column either upward or downward. You need to write a function that will take the 2D array and combination of the safe as an 1D array of length 5 and unlock the safe. To unlock the safe you need to make the **middle row (4th row as rows start from 0) same as the input 1D array**.

Note that the values of each column are unique and in the range of 1 to 9. To equate the middle row with the given combination (1D array), you need to perform **column rotation operation** (Just like an original safe). You may write and use helper functions.

The space complexity of your solution should be $O(1)$ that means you cannot create any new array. **Finally, return the resulting array. In addition, write the Time complexity of your solution in terms of Big O notation.**

Input Array	Returned Array	Explanation																																																																																															
<table><tr><td>2</td><td>8</td><td>9</td><td>6</td><td>7</td></tr><tr><td>4</td><td>2</td><td>5</td><td>8</td><td>5</td></tr><tr><td>6</td><td>7</td><td>1</td><td>4</td><td>3</td></tr><tr><td>9</td><td>6</td><td>7</td><td>2</td><td>9</td></tr><tr><td>7</td><td>9</td><td>3</td><td>5</td><td>6</td></tr><tr><td>8</td><td>1</td><td>6</td><td>9</td><td>2</td></tr><tr><td>5</td><td>4</td><td>2</td><td>7</td><td>1</td></tr><tr><td>3</td><td>3</td><td>8</td><td>1</td><td>4</td></tr><tr><td>1</td><td>5</td><td>4</td><td>3</td><td>8</td></tr></table> combination: <table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	2	8	9	6	7	4	2	5	8	5	6	7	1	4	3	9	6	7	2	9	7	9	3	5	6	8	1	6	9	2	5	4	2	7	1	3	3	8	1	4	1	5	4	3	8	1	2	3	4	5	<table><tr><td>7</td><td>4</td><td>9</td><td>1</td><td>1</td></tr><tr><td>8</td><td>3</td><td>5</td><td>3</td><td>4</td></tr><tr><td>5</td><td>5</td><td>1</td><td>6</td><td>8</td></tr><tr><td>3</td><td>8</td><td>7</td><td>8</td><td>7</td></tr><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr><tr><td>2</td><td>7</td><td>6</td><td>2</td><td>3</td></tr><tr><td>4</td><td>6</td><td>2</td><td>5</td><td>9</td></tr><tr><td>6</td><td>9</td><td>8</td><td>9</td><td>6</td></tr><tr><td>9</td><td>1</td><td>4</td><td>7</td><td>2</td></tr></table>	7	4	9	1	1	8	3	5	3	4	5	5	1	6	8	3	8	7	8	7	1	2	3	4	5	2	7	6	2	3	4	6	2	5	9	6	9	8	9	6	9	1	4	7	2	The returned array is rotated column wise in such a way so that the 4th row and the combination array have the same sequence of values. Column wise rotation may be done upward or downward. No need to implement both.
2	8	9	6	7																																																																																													
4	2	5	8	5																																																																																													
6	7	1	4	3																																																																																													
9	6	7	2	9																																																																																													
7	9	3	5	6																																																																																													
8	1	6	9	2																																																																																													
5	4	2	7	1																																																																																													
3	3	8	1	4																																																																																													
1	5	4	3	8																																																																																													
1	2	3	4	5																																																																																													
7	4	9	1	1																																																																																													
8	3	5	3	4																																																																																													
5	5	1	6	8																																																																																													
3	8	7	8	7																																																																																													
1	2	3	4	5																																																																																													
2	7	6	2	3																																																																																													
4	6	2	5	9																																																																																													
6	9	8	9	6																																																																																													
9	1	4	7	2																																																																																													

Python Notation	Java Notation
<pre>def unlockSafe(mat, com): # To Do</pre>	<pre>public int[][] unlockSafe(int[][] mat, int[] com){ // To Do }</pre>

20.

Question 1: CO1 [8 Points]

You're working as a terrain analyst for a Mars exploration rover. The rover receives a grid (matrix) of terrain height values scanned from orbit. However, due to occasional solar interference, some readings show spikes (noisy values) that are unrealistically high and should be ignored.

Your task is to complete the following function/method to help the rover find a local minimum — a position where the height is lower than its adjacent neighbors (top, bottom, left, right), excluding any cells where the height is above a certain noise threshold.

To help guide the rover better, **if multiple valid local minima exist, choose the last one from the bottom right of the matrix.**

A local minimum is a cell strictly less than its up, down, left, and right neighbors (4 directions).

Ignore any cell or neighbor that is above the given noise threshold.

Input Array	Returned Array	Explanation
<pre>matrix = [[22, 21, 1000, 23], [25, 17, 21, 1000], [1000, 19, 18, 22], [21, 22, 1000, 20]] noise_threshold = 100</pre>	Local minimum found at: (2, 2)	• (2, 2) has value 18. • Its neighbors: 21 (up), 1000 (down, ignored), 19 (left), 22 (right). • 18 is less than all valid neighbors and the bottom rightmost item

Python Notation	Java Notation
<pre>def localMinima(matrix, threshold): # To DO</pre>	<pre>public static void localMinima (int[][] matrix, int threshold){ //To DO }</pre>

21.

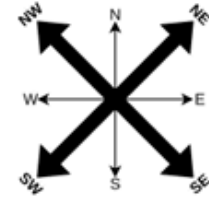
You are exploring an **energy grid matrix** using a **direction map matrix**, both of which are of an **m x n** shape.

- **grid[x][y]** – contains **integer** energy values where x , y are the indices.
- **directions[x][y]** – contains **String** direction symbols: "NE", "NW", "SE", "SW".

Each cell of the **directions matrix** tells you in which direction the energy comes to that specific coordinate of the **grid matrix**. Meaning, for the sample example below the (0,1) cell is getting energy from all the cells which are "South East" of itself where the values are 7,12. So, the (0,1) cell's energy is 2+7+12=21.

Your task is to compute how much total energy each block absorbs by following the direction shown on the direction map. So, write a function/method **blockEnergySum()** that computes a new **Energy Matrix** where:

- For each cell in the grid matrix, start from its own position.
- Look at its direction symbol (i.e "SE" for [0,1]) in the directions matrix. Keep moving in that diagonal direction in the grid matrix until you exit the grid.
- While moving, you sum up all visited cell values, including the starting cell of each cell.
- Do this for all the cells of the grid matrix. Lastly, **return the newly computed Energy Matrix** where each of the cells contain the energy summations of each respective cell of the grid matrix.



Note: The space complexity of your solution must not exceed **O(m*n)**.

Given Matrices								Returned Energy Matrix																			
grid =				directions =				<table><tr><td>34</td><td>21</td><td>18</td><td>4</td></tr><tr><td>5</td><td>15</td><td>11</td><td>8</td></tr><tr><td>9</td><td>23</td><td>19</td><td>21</td></tr><tr><td>13</td><td>23</td><td>15</td><td>16</td></tr></table>				34	21	18	4	5	15	11	8	9	23	19	21	13	23	15	16
34	21	18	4																								
5	15	11	8																								
9	23	19	21																								
13	23	15	16																								
1	2	3	4	"SE"	"SE"	"SW"	"NE"																				
5	6	7	8	"SW"	"SW"	"NE"	"SE"																				
9	10	11	12	"NW"	"SW"	"NE"	"NW"																				
13	14	15	16	"SE"	"NW"	"SW"	"NE"																				
Explanation																											
[0,0]⇒ SE: includes its own value 1 then SE path adds 6,11,16 ⇒ $1+(6+11+16) = 34$ [0,1]⇒ SE: includes its own value 2 then SE path adds 7,12 ⇒ $2+(7+12) = 21$ [0,2]⇒ SW: includes its own value 3 then SW path adds 6,9 ⇒ $3+(6+9) = 18$ [0,3]⇒ NE: includes its own value 4 but moving NE immediately exits the grid, so no additional cells are added ⇒ total energy = 4 which is shown in the corresponding index of returned Energy Matrix. [1,0]⇒ SW: same as [0,3] since the "SW" direction also exits the grid ⇒ total energy = 5. [1,1]⇒ SW: includes its own value 6, then SW path adds 9 ⇒ $6+(9) = 15$. Other cells follow the same process: summing the cell's own value & cells along its diagonal direction.																											
Java Method Signature								Python Function Signature																			
<pre>static int[][] blockEnergySum(int[][] grid, String[][] directions){ //TODO }</pre>								<pre>def blockEnergySum(grid , directions) #TODO</pre>																			